

ELI — Grounding And Evidence

ELI v2.4.80

September 2026

Contents

ELI Grounding & Evidence Layer	1
The core idea	1
Components	2
How it fits the pipeline	3
Honest assessment	3
Behaviours that keep answers grounded	4
Deterministic reports are surfaced verbatim; examiner correctness	4
News topic deepen (grounded, not guessed)	4
Challenging a grounded answer must re-check it, not confabulate a new one	4
History is not discarded before the budgeter sees it	5

ELI Grounding & Evidence Layer

Updated for v2.4.72. Quick mode still returns verbatim for deterministic introspection; all CHAT modes now pass through the gradient orchestrator first.

The anti-confabulation system — a deterministic evidence scaffold wrapped around the probabilistic model. This is ELI’s most distinctive subsystem and the part closest to genuinely frontier. Spans `eli/runtime/` and `eli/cognition/`.

The core idea

For “grounded” actions (status, runtime, memory, identity, control), ELI does **not** trust the LLM to state facts. It (1) gathers deterministic evidence from the live system, (2) can answer directly from that evidence without the LLM at all, and (3) when the LLM does generate, validates the output against the evidence and rejects/repairs contradictions. The LLM becomes a phraser, not a source.

Correction (2026-06-08): the “bypass” is PARTIAL and MODE-GATED — not a blanket bypass. Earlier wording here and in `what_eli_is.md` (“bypasses the language model entirely”, “without the LLM at all”) overstated it. The running reality, verified against the engine: the deterministic verbatim return fires for a **subset** of actions and is **mode-gated** — - a small **_verbatim_always_actions** set (deep introspection like `EXPLAIN_MEMORY_RUNTIME`) returns verbatim in **every** reasoning mode; - **_deterministic_direct_payload_actions** (status/reports + router-fast/command actions: `DATE/TIME`, `VOLUME`, `WRITE_NOTE`, window/media/file ops, `NEWS/MORNING_REPORT`, self-report ...) return **verbatim in quick mode** and **synthesise in non-quick** modes (`engine._bypass_persona + _is_grounding_control_nonquick`); - everything else is the model’s to phrase.

So the model IS in the loop for most conversational turns; the “phraser, not a source” principle holds where the deterministic path actually fires, not universally. A 2026-06-08 fix corrected the membership: the command actions were in the wrong set, so quick mode re-synthesised

them, corrupting results (“Wrote note”→“Bought note”) and adding latency. They are now verbatim in quick mode, synthesised in others.

Components

`runtime/deterministic_grounding_gate.py` (3.4k lines)

The deterministic renderer. `render_action(action, args, user_input, mode_label)` produces a grounded answer for control/status actions directly from runtime data (settings, runtime snapshot, DB counts, GPU line) — bypassing the model. `install(CognitiveEngine)` wires it into the engine. `_eli_v14_runtime_data()` assembles the live config block (model_path, n_ctx, gpu_layers, ...).

Structure. `render_action` is one explicit ordered pipeline: each layer is a function that takes the layer below it as an argument, composed by `_stack(...)` into `_RENDER_PIPELINE`. An action no layer handles returns a JSON surface missing `_deterministic_renderer` rather than a made-up answer. The layer order is pinned by a characterisation test that compares the rendered output of every audited action.

`_eli_last_response_confidence_v2()` (called for “how confident are you in your last answer”) used to return a fixed template string regardless of what the last response actually was — a confident-sounding self-assessment that inspected nothing. It now reuses `eli.runtime.last_trace.load_last_trace()` + `control_contracts._trace_text()`, the same real per-turn trace mechanism already used correctly for “what was your last message,” and says plainly when no trace is available rather than fabricating an answer.

`runtime/control_contracts.py` (1,232 lines)

The deterministic control path: - `is_control_action / route_control_text` — recognise control/status intents. - `build_control_evidence(engine, action, args, ...)` — gather the evidence packet (runtime paths, DB state, bus result, trace). - **`output_violates_evidence(text, evidence_text)`** — the gate that returns True when LLM output contradicts/omits the evidence; the engine uses this to reject a hallucinated answer. Seven concrete runtime terms (gpu layers, batch size, context size, cpu threads, model path, user/agent database) used to be blanket-exempted from this check, because evidence renders them under a machine-style key (n_gpu_layers, model_path...) while prose uses the human phrase — a plain substring match would reject a correct answer over spelling alone. That exemption also let the model state those exact values with *no* evidence backing at all. `_CONCRETE_TERM_ALIASES` now maps each term to every real spelling it appears under, closing the hole without reintroducing the false positives. - `compact_evidence_answer / finalise_control_result` — assemble the final grounded response.

Synthesis is validated against its evidence

`cognition/output_governor.validate_against_evidence` also rejects any figure or model designation (H100, 512 GB, 32K) that the evidence does not contain, allowing for unit conversion; the non-Quick compact synthesis path uses it and falls back to the deterministic evidence text. `contracts/grounded_control.py` owns which actions never fall back to a clarifying question.

`runtime/evidence_ledger.py` (603 lines)

A persistent SQLite ledger of evidence events: `record_event`, `recent_events`, `repeated_event_signals` (detect recurring issues over N days), `status_evidence`, `artifact_snapshot`. Gives ELI a durable, queryable record of what actually happened.

`runtime/evidence_arbitration.py` (193 lines)

`EvidenceItem` + `arbitrate_evidence(limit)` + `build_evidence_context_text` — scores and merges competing evidence into a single context block. Pairs with the agent-bus confidence aggregation

(`_score_tool_result`), which scores a tool result with no recorded "ok" field as unverified (same low score as a confirmed failure) rather than defaulting a missing outcome to success.

Retrieval provenance

Memory hits reach the prompt through the shared turn retrieval in `memory/retrieval.py`. Each recalled line carries its event date and status (`memory_provenance.format_grounding_memory_line`) and the orchestrator adds a retrieval diagnostics block (what was searched, how many items, which time window) so ELI can say truthfully what its search did.

cognition/output_governor.py (1,681 lines)

Post-generation governor: `govern_output(text, is_grounding)`, `normalize_assistant_text`, `validate_against_evidence`, plus a family of drift-repair functions — `strip_generic_ai_identity_drift` (kills "As an AI language model..."), `repair_local_persona_drift`, `repair_self_user_confusion` (fixes the model conflating itself with the user), `clean_response_style`. The last line of defence before text reaches the user.

cognition/grounded_status.py (648 lines)

`direct_grounding_answer(user_text)` — fully deterministic answers for identity / memory-inventory / status questions, assembled from the DBs (`format_user_identity`, `format_memory_inventory`, table distributions). No LLM.

runtime/grounding_remediation.py (1.7k lines)

The failure→repair loop. When an action fails (app won't open, path missing, browser/IDE absent), it: `diagnose_app/path/browser/ide` → `build_repair_plan` → `offer_for_result` ("want me to install X?") → `handle_confirmation` (consumes yes/no) → `execute_pending_plan`. Stateful pending-repair tracking + `explain_last_failure`. This is what makes failures conversational and recoverable instead of dead ends.

How it fits the pipeline

1. Router classifies action. Control/grounding actions enter the deterministic path.
2. `build_control_evidence` gathers facts.
3. PHASE45 deterministic bypass: for many status actions `render_action` / `direct_grounding_answer` answer **without** the LLM.
4. If the LLM generates, `output_violates_evidence` + `validate_against_evidence` gate it; `govern_output` cleans drift.
5. `grounding_remediation` handles action failures with an offer/confirm loop.
6. `evidence_ledger` records the event for future `repeated_event_signals`.

Honest assessment

- **Strong (genuinely):** very few local-LLM projects build a deterministic evidence layer at all, let alone one this thorough — direct-answer bypass, output-vs-evidence rejection, drift repair, a persistent ledger, and a conversational remediation loop. This is the subsystem that most justifies the "frontier" label.
- **Weak:**
 1. The gate file is still 3.4k lines with many version-named helpers (`_eli_v9_*`, `_eli_v14_*`); the layers are ordered explicitly now, but the helpers wait to be folded into fewer, named modules.
 2. **Overlapping surfaces** — `grounding_status`, `control_contracts`, `deterministic_grounding_gate`, and the `runtime/*_response` / `*_surface` modules all render grounded answers with partial overlap. The boundaries between "who renders the final grounded string" are blurry.
 3. Confidence/grounding is heuristic (additive score + threshold + `output_violates_evidence`), not a formal proof — fine, but it's a gate, not a guarantee.

Behaviours that keep answers grounded

- **Self-patch only patches what it can ground.** `generate_code_patch` extracts the target file from the error traceback; when a failure has no in-project file (e.g. an HTTP/connection error, “No commands to run”), it used to ask the model anyway, which **invented** a path (phantom `api_client.py`) that then failed to apply. It now returns `no_groundable_file` and routes the failure to goal-autogenesis / self-heal instead; when a file IS grounded it is **pinned** in the prompt so the model can’t drift to a different path.
- **Banter guard:** the `llm_intent` resolver occasionally maps playful input to a generative action; the engine now downgrades `GENERATE_SCRIPT/PROJECT/DOCUMENT/CODE_SOLVE` to `CHAT` when the text carries no create-intent keyword (real “write me a script” is preserved).
- **Compact grounded synthesis carries a VOICE primer** (pulled live from the canonical persona) so factual/introspection answers sound like ELI without losing the EXACT-FACTS contract that pins every number/path/table/DB to the evidence.

Deterministic reports are surfaced verbatim; examiner correctness

- **Verbatim guard for deterministic grounded reports.** `_get_chat_response` now returns the `EXAMINE_CODE` tiered report and the `FILE_AUDIT` file-count `VERBATIM` when they appear in the evidence, and surfaces the `FIX_FILE` success event as a real outcome — instead of letting the quick chat path re-narrate them. This killed three live confabulation classes: `EXAMINE_CODE` inventing “findings” that were actually existing code comments; `FILE_AUDIT` inventing files-with-bugs that don’t exist; `FIX_FILE` narrating “I cannot fix” after it had already written the file.
- **Examiner correctness (`code_examiner.py`).** Tier-2 now drops pyflakes star-import noise (“may be undefined, or defined from star imports” — ~800 false positives on the GUI → 0). Tier-3 reviews the `WHOLE` file in line-correct windows (was only the first ~`MAX_FILE_CHARS`, so god-files were ~90% unreviewed), each source line prefixed with its `REAL` number, and rejects any finding citing a line outside the shown window (kills the “`engine.py:132` undefined ‘`ov`’” hallucination class — line 132 is or `result.get("error")`).
- **Routing:** “is there any issues with the files?” / “check the code for bugs” now route to `EXAMINE_CODE` (the real examiner), not `FILE_AUDIT`; `fix-intent` (“fix the bugs in `foo.py`”) stays `FIX_FILE`.

News topic deepen (grounded, not guessed)

After a `NEWS_FETCH` briefing, a follow-up like “**go deeper into the transformer story**” must not route to open `CHAT` and let the model invent details. The engine (`_try_news_topic_deepen_reroute` in `engine.py`) detects `extract_deepen_topic()` within a short window of the last news command and **re-routes to `NEWS_FETCH` with that topic** — same live fetch path as the original briefing.

If the user complains that ELI guessed (`is_news_fetch_complaint`), the kernel recovers the deepen topic from recent conversation (`recover_recent_deepen_topic`) and runs `NEWS_FETCH` again instead of falling through to web-escalation hedge.

Challenging a grounded answer must re-check it, not confabulate a new one

After a grounded action (`RUNTIME_AUDIT`, `RUNTIME_STATUS`, `GPU_STATUS`, `MEMORY_STATUS`, and ~20 other status/diagnostic actions), a detail or challenge follow-up must rebind to the *same* action for a fresh evidence pull, not fall through to open `CHAT` — that lets the model free-associate over whatever it remembers saying, and a skeptical push (“you sure about that?”) can make it retract a correct answer and invent a wrong one instead (`_eli_phase19_rebind_grounding_followup` in `engine.py`). Two ways this can silently not fire:

- the follow-up’s wording doesn’t match `_ELI_PHASE19_CHALLENGE_FOLLOWUP_RX / ..._DETAIL...RX` (“and you think the maths checks out??” wasn’t covered until it was reproduced live — `GPU_STATUS` correctly explained requested-vs-effective GPU layers, then confabulated a different, wrong breakdown when challenged in exactly this phrasing);
- the prior action isn’t in `_ELI_PHASE19_GROUNDED_FOLLOWUP_ACTIONS` — `GPU_STATUS` and most of its sibling status/diagnostic actions were missing from this set entirely, despite publishing `grounded=True`, `evidence_used=True` through the identical `_publish_last_response_meta` call site as the actions that were already covered.

`tests/test_phase19_grounded_followup_truth_lock.py` pins both the regex and the action set against the live failure.

History is not discarded before the budgeter sees it

The old memory-evidence module capped recent-history pulls silently; it was removed and shared turn retrieval (`memory/retrieval.py`) replaced it. The user-facing setting `cog.mem_recent_turns` (default 24, maximum 80) and `cognition/context_budget.py` now decide how much history and memory enter the prompt, and a recall question keeps a floor of a third of the window for memory context.